

CLASE 4

Templates, herencia y contexto

El HTML saldrá de la View y se convertirá en una interfaz reutilizable

View → Contexto → Template

La ruta funciona, pero la presentación quedó atrapada

- La URL dirige correctamente hacia la View.
- La View decide la respuesta y el estado HTTP.
- El HTML está escrito dentro del código Python.

Necesitamos conservar la lógica y trasladar la presentación a archivos HTML.

¿Qué responsabilidad debería mantener la View y cuál debería asumir el Template?

La View coordina la solicitud y prepara los datos; el Template se encarga de presentarlos como HTML.

Hoy construiremos páginas HTML reutilizables

OBJETIVO DE APRENDIZAJE

Implementar Templates de Django con herencia, contexto y navegación por nombres de URL, separando la presentación de las responsabilidades de la View.

Al finalizar podrás...

- 01 Configurar y ubicar correctamente los Templates.
- 02 Crear base.html y una página hija.
- 03 Enviar y representar datos desde una View.
- 04 Comprobar rutas, navegación y contenido dinámico.

HttpResponse sirve para comenzar; render separa responsabilidades

HTML EN HttpResponse

- Útil para una prueba mínima.
- Mezcla Python y presentación.
- Repite estructura entre páginas.
- Escala con dificultad.

TEMPLATE CON render

- HTML en archivos especializados.
- Datos enviados mediante contexto.
- Estructura común reutilizable.
- Interfaz más fácil de mantener.

La View conserva la coordinación; el Template recibe datos y construye la presentación.

Django Architecture: Refactorización de Vistas

Evolución técnica de `HttpResponse` directo hacia `render()` con Plantillas



Aclaración Técnica: `HttpResponse` NO es un error ni un antipatrón

`HttpResponse` es el objeto HTTP fundamental de Django. La función `render()` devuelve un objeto `HttpResponse` internamente. La decisión técnica de refactorizar no busca corregir una falla, sino aplicar el principio de **Separación de Responsabilidades (SoC)**, extrayendo el marcado gráfico del código fuente en Python.

ENFOQUE ACOPLADO (A EVITAR)

Antes: HTML Embebido en la Vista

myapp/views.py

```
from django.http import HttpResponse

def detalle_producto(request, id):
    prod = Producto.objects.get(pk=id)
    # Mezcla de lógica de negocio + maquetación HTML
    html = f"<h1>{prod.nombre}</h1><p>${prod.precio}</p>"
    return HttpResponse(html)
```

⚠ Inconvenientes del Acoplamiento Directo:

- Dificulta la lectura y mantenimiento a medida que el HTML crece.
- Sin soporte de autocompletado ni formateo nativo de HTML.
- Exige sanitización manual de variables para prevenir XSS.

ENFOQUE DESACOPLADO (RECOMENDADO)

Después: Vista Desacoplada + Plantilla

myapp/views.py (La decisión PERMANECE en Python)

```
from django.shortcuts import render

def detalle_producto(request, id):
    prod = Producto.objects.get(pk=id)
    contexto = {'producto': prod}
    return render(request, 'detalle.html', contexto)
```

myapp/templates/detalle.html (El HTML SE MUEVE aquí)

```
<h1>{{ producto.nombre }}</h1>
<p>${{ producto.precio }}</p>
```

✨ Beneficios de la Separación:

- **Lógica (views.py):** Decide qué datos obtener, procesar y enviar.
- **Presentación (detalle.html):** Define la estructura e interfaz gráfica.

1. Mantenibilidad y Limpieza

El código Python se mantiene enfocado exclusivamente en la lógica de control. Alterar estilos, CSS o estructura no toca la lógica del servidor.

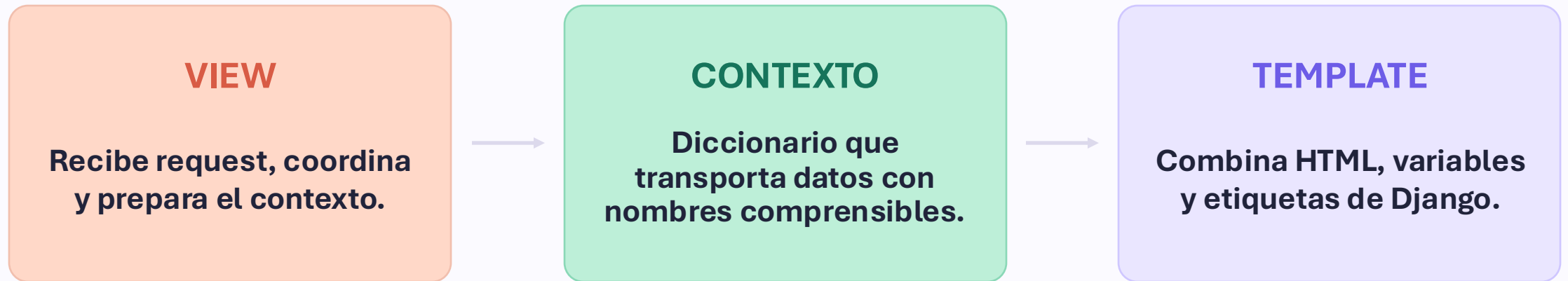
2. Especialización de Trabajo

Facilita la colaboración: desarrolladores Frontend editan archivos `.html` con sus herramientas, sin intervenir en el código Python de Backend.

3. Motor de Plantillas y Seguridad

Aprovecha herencia de plantillas (`{% extends %}`), filtros reutilizables y **escape automático XSS** integrado en Django.

El recorrido mantiene tres responsabilidades distintas



`render(request, template, contexto)` produce una respuesta HTML a partir de datos preparados por la View.

Paso 1: verifica el proyecto y prepara templates

```
git status

source .venv/Scripts/activate

python manage.py check

# Estructura que crearemos
templates/
├── base.html
├── dispositivos/
│   ├── inicio.html
│   └── catalogo.html
```

Observa

- Trabaja desde la carpeta que contiene manage.py.
- No versiones .venv ni archivos temporales.
- El subdirectorio evita colisiones de nombres.

Paso 2: agrega la carpeta global en settings.py

```
# config/settings.py
TEMPLATES = [
    {
        "BACKEND":
        "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        # ... opciones generadas por Django
    },
]

python manage.py check
```

Observa

- BASE_DIR representa la raíz del proyecto.
- DIRS incorpora templates/ como ubicación común.
- APP_DIRS conserva la búsqueda dentro de aplicaciones.

Paso 3: base.html define la estructura compartida

```
<!-- templates/base.html -->
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>{% block title %}EcoEnergy{% endblock %}</title>
</head>
<body>
  <nav>
    <a href="{% url 'dispositivos:inicio' %}">Inicio</a>
    <a href="{% url 'dispositivos:catalogo' %}">Dispositivos</a>
  </nav>
  <main>{% block content %}{% endblock %}</main>
</body>
</html>
```

Observa

- block marca zonas que una página hija puede reemplazar.
- url usa nombres de ruta en lugar de direcciones fijas.
- La navegación se define una sola vez.

Paso 4: una página hija completa los bloques

```
<!-- templates/dispositivos/inicio.html -->
{% extends "base.html" %}

{% block title %}Inicio | EcoEnergy{% endblock %}

{% block content %}
    <h1>{{ sistema }}</h1>
    <p>{{ mensaje }}</p>
    <p>Curso: {{ asignatura }}</p>
{% endblock %}
```

Observa

- extends debe aparecer al inicio del Template.
- {{ variable }} representa un valor del contexto.
- La página hija no repite html, head, body ni nav.

Paso 5: la View prepara contexto y utiliza render

```
# dispositivos/views.py
from django.shortcuts import render

def inicio(request):
    contexto = {
        "sistema": "EcoEnergy",
        "mensaje": "Monitoreo energético responsable",
        "asignatura": "Programación Back End",
    }

    return render(
        request,
        "dispositivos/inicio.html",
        contexto,
    )
```

Observa

- La View conserva las decisiones y prepara los datos.
- Cada clave queda disponible como variable.
- render devuelve una respuesta HTML.

Paso 6: el contexto también puede transportar una colección

```
# dispositivos/views.py

def catalogo(request):
    dispositivos = [
        {"nombre": "Medidor inteligente", "estado": "Activo"},
        {"nombre": "Sensor de temperatura", "estado": "Activo"},
        {"nombre": "Climatizador", "estado": "Revisión"},
    ]
    return render(
        request,
        "dispositivos/catalogo.html",
        {"dispositivos": dispositivos},
    )

# dispositivos/urls.py

path("dispositivos/", views.catalogo, name="catalogo")
```

Observa

- Los datos temporales permanecen en Python.
- El Template recibe la colección con una clave.
- La ruta posee un name utilizado por la navegación.

Paso 7: el Template recorre y presenta los datos

```
<!-- templates/dispositivos/catalogo.html -->
{% extends "base.html" %}

{% block title %}Dispositivos | EcoEnergy{% endblock %}

{% block content %}
  <h1>Dispositivos</h1>
  <ul>
    {% for dispositivo in dispositivos %}
      <li>{{ dispositivo.nombre }} - {{ dispositivo.estado }}</li>
    {% empty %}
      <li>No existen dispositivos disponibles.</li>
    {% endfor %}
  </ul>
{% endblock %}
```

Observa

- for repite la presentación por cada elemento.
- La notación punto accede a valores del diccionario.
- empty cubre el caso de una colección vacía.

El error revela qué parte del recorrido revisar

SEÑAL

- TemplateDoesNotExist.
- NoReverseMatch.
- La variable aparece vacía.
- Error en etiqueta de Template.

PRIMERA COMPROBACIÓN

- DIRS, carpeta y nombre solicitado.
- app_name y name de la URL.
- Clave del contexto y variable.
- Llaves, porcentajes y cierres.

Depura en orden: configuración → ruta → View/contexto → Template.

Laboratorio: transforma una ruta en una página reutilizable

SITUACIÓN

Convierte una respuesta de texto de tu Proyecto Integrado o de EcoEnergy en una página hija que herede base.html y represente datos enviados por una View.

1

Elige una ruta distinta del catálogo demostrado.

2

Crea o reutiliza base.html con navegación.

3

Crea un Template hijo dentro de un subdirectorio.

4

Envía al menos tres datos mediante contexto.

Entrega individual: URL funcional, Template con herencia, contexto comprobado y enlace al commit.

La IA ayuda a documentar Templates sin inventar

Actualiza el README.md de mi proyecto Django con información comprobada.

Templates: [ARCHIVOS]
Plantilla base: [BASE]
Rutas funcionales: [URLS]
Claves de contexto: [CLAVES]

Incluye estructura, ejecución y prueba de navegación.
No inventes archivos, comandos ni funcionalidades.
Si falta información, pregúntame antes de redactar.

Observa

- Entregar nombres y rutas reales.
- Comprobar la ejecución antes de documentarla.
- Explicar qué datos prepara cada View.
- Excluir secretos, tokens y datos personales.

Ticket de salida: explica cómo llega un dato al HTML

TICKET DE SALIDA

¿Qué recorrido realiza la variable sistema desde la View hasta aparecer en inicio.html?

Responde brevemente

1. Respuesta individual en cuatro pasos + enlace al commit.

PRÓXIMA CLASE

Clase 5: estructuras Python, archivos JSON y datos representados en Templates.